

Threads

Common Concurrency Bugs

CS311 Operating System, Fall 2020

Concurrency bugs

- Main classification:
 - Non-deadlock bugs
 - Deadlock bugs

Non-deadlock bugs – atomicity violation

```
1  Thread 1::
2  if (thd->proc_info) {
3      ...
4      fputs(thd->proc_info, ...);
5      ...
6  }
7
8  Thread 2::
9  thd->proc_info = NULL;

1 pthread_mutex_t proc_info_lock = PTHREAD_MUTEX_INITIALIZER;
2
3  Thread 1::
4  pthread_mutex_lock(&proc_info_lock);
5  if (thd->proc_info) {
6      ...
7      fputs(thd->proc_info, ...);
8      ...
9  }
10 pthread_mutex_unlock(&proc_info_lock);
11
12 Thread 2::
13 pthread_mutex_lock(&proc_info_lock);
14 thd->proc_info = NULL;
15 pthread_mutex_unlock(&proc_info_lock);
```

Non-deadlock bugs – order violation

```
1  Thread 1::
2  void init() {
3      ...
4      mThread = PR_CreateThread(mMain, ...);
5      ...
6  }
7
8  Thread 2::
9  void mMain(...) {
10     ...
11     mState = mThread->State;
12     ...
13 }
```

```
1  pthread_mutex_t mtLock = PTHREAD_MUTEX_INITIALIZER;
2  pthread_cond_t  mtCond = PTHREAD_COND_INITIALIZER;
3  int mtInit
4      = 0;
5
6  Thread 1::
7  void init() {
8      ...
9      mThread = PR_CreateThread(mMain, ...);
10
11     // signal that the thread has been created...
12     pthread_mutex_lock(&mtLock);
13     mtInit = 1;
14     pthread_cond_signal(&mtCond);
15     pthread_mutex_unlock(&mtLock);
16     ...
17 }
18
19 Thread 2::
20 void mMain(...) {
21     ...
22     // wait for the thread to be initialized...
23     pthread_mutex_lock(&mtLock);
24     while (mtInit == 0)
25         pthread_cond_wait(&mtCond, &mtLock);
26     pthread_mutex_unlock(&mtLock);
27
28     mState = mThread->State;
29     ...
30 }
```

Dead lock bugs

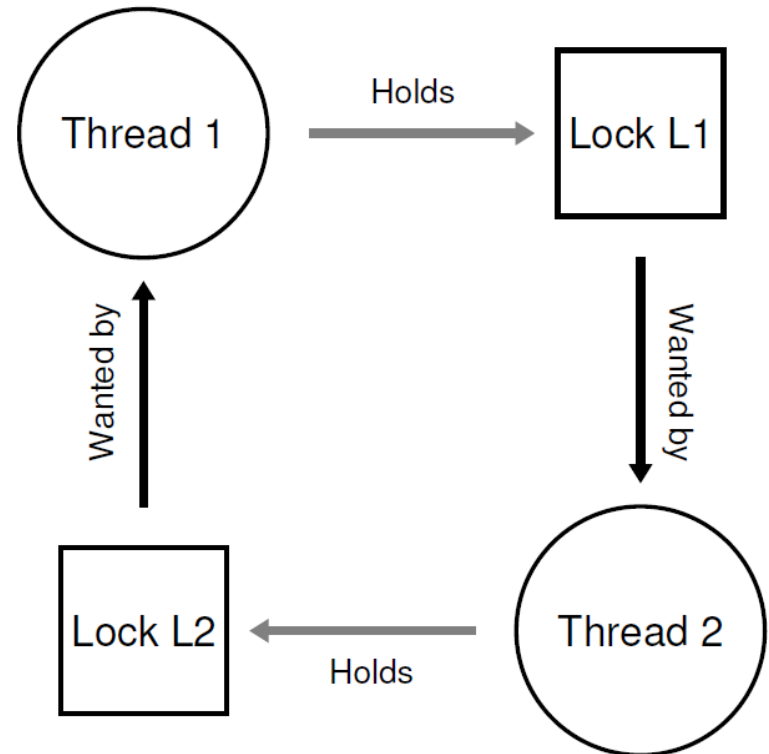
Thread 1:

```
pthread_mutex_lock(L1);  
pthread_mutex_lock(L2);
```

Thread 2:

```
pthread_mutex_lock(L2);  
pthread_mutex_lock(L1);
```

- Why do deadlocks occur?



Encapsulation

```
Vector v1, v2;
```

```
v1.AddAll(v2);
```

```
v2.AddAll(v1)
```

Conditions for deadlock

- Mutual exclusion
- Hold-and-wait
- No preemption
- Circular wait

Deadlock prevention – circular wait

- Impose ordering when acquiring locks

Deadlock prevention – hold-and-wait

- All resources should be acquired at once, atomically!

```
1  pthread_mutex_lock(prevention);    // begin lock acquisition
2  pthread_mutex_lock(L1);
3  pthread_mutex_lock(L2);
4  ...
5  pthread_mutex_unlock(prevention);  // end
```

Deadlock prevention – no preemption

- Release acquired resources if unsuccessful

```
1  top:
2      pthread_mutex_lock(L1);
3      if (pthread_mutex_trylock(L2) != 0) {
4          pthread_mutex_unlock(L1);
5          goto top;
6      }
```

Deadlock prevention – mutual exclusion

- Use atomic instructions directly if we can:

```
1  int CompareAndSwap(int *address, int expected, int new) {
2      if (*address == expected) {
3          *address = new;
4          return 1; // success
5      }
6      return 0; // failure
7  }
```

```
1  void AtomicIncrement(int *value, int amount) {
2      do {
3          int old = *value;
4      } while (CompareAndSwap(value, old, old + amount) == 0);
5  }
```

Contd.

```
1 void insert(int value) {
2     node_t *n = malloc(sizeof(node_t));
3     assert(n != NULL);
4     n->value = value;
5     n->next = head;
6     head = n;
7 }
```

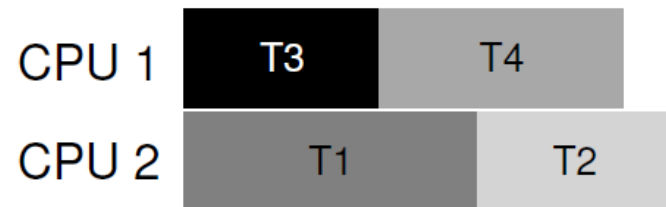
```
1 void insert(int value) {
2     node_t *n = malloc(sizeof(node_t));
3     assert(n != NULL);
4     n->value = value;
5     pthread_mutex_lock(listlock); // }
6     n->next = head;
7     head = n;
8     pthread_mutex_unlock(listlock); // {
9 }
```

```
1 void insert(int value) {
2     node_t *n = malloc(sizeof(node_t));
3     assert(n != NULL);
4     n->value = value;
5     do {
6         n->next = head;
7     } while (CompareAndSwap(&head, n->next, n) == 0);
8 }
```

Deadlock avoidance via scheduling

	T1	T2	T3	T4
L1	yes	yes	no	no
L2	yes	yes	yes	no

- If scheduler has knowledge about threads



Detect and recover

- Used in databases, etc.